

Comparaison expérimentale des algorithmes de chiffrement symétrique

Melissa Moya

Présenté à Habib Louafi • TN3



```
TRIALS = 500

def measure_avalanche_at_rounds(n_rounds: int) -> float:
    key = os.urandom(8)
    total_bits = 64
    scores = []

    for _ in range(TRIALS):
        block = os.urandom(8)
        ref_ct = des_encrypt_n_rounds(block, key,
                                        n_rounds)
        flip_byte = secrets.randbelow(8)
        flip_bit = secrets.randbelow(8)
        modified = bytearray(block)
        modified[flip_byte] ^= (1 << flip_bit)
        mod_ct = des_encrypt_n_rounds(bytes(modified),
                                        key, n_rounds)
        diff_bits = sum(bin(a ^ b).count("1") for a, b in zip(ref_ct, mod_ct))
        scores.append(diff_bits / total_bits)

    return sum(scores) / len(scores)
```

```
TRIALS = 500

def measure_avalanche_at_rounds(n_rounds: int) -> float:
    key = os.urandom(8)
    total_bits = 64
    scores = []

    for _ in range(TRIALS):
        block = os.urandom(8)
        ref_ct = des_encrypt_n_rounds(block, key,
                                        n_rounds)
        flip_byte = secrets.randbelow(8)
        flip_bit = secrets.randbelow(8)
        modified = bytearray(block)
        modified[flip_byte] ^= (1 << flip_bit)
        mod_ct = des_encrypt_n_rounds(bytes(modified),
                                        key, n_rounds)
        diff_bits = sum(bin(a ^ b).count("1") for a, b in zip(ref_ct, mod_ct))
        scores.append(diff_bits / total_bits)

    return sum(scores) / len(scores)
```

```
TRIALS = 500

def measure_avalanche_at_rounds(n_rounds: int) -> float:
    key = os.urandom(8)
    total_bits = 64
    scores = []

    for _ in range(TRIALS):
        block = os.urandom(8)
        ref_ct = des_encrypt_n_rounds(block, key,
                                        n_rounds)
        flip_byte = secrets.randbelow(8)
        flip_bit = secrets.randbelow(8)
        modified = bytearray(block)
        modified[flip_byte] ^= (1 << flip_bit)
        mod_ct = des_encrypt_n_rounds(bytes(modified),
                                        key, n_rounds)
        diff_bits = sum(bin(a ^ b).count("1") for a, b in zip(ref_ct, mod_ct))
        scores.append(diff_bits / total_bits)

    return sum(scores) / len(scores)
```



Plan de **présentation**

01

Nature du système et objectifs

Système de comparaison de chiffrement symétrique: périmètre, architecture et critères d'évaluation.

02

Protocole expérimental et environnement

Protocole unique, paramètres contrôlés et exécution identique sur x86 et ARM pour garantir l'équité.

03

Validation cryptographique — KAT

Conformité des implémentations vérifiée par vecteurs NIST/RFC avant toute analyse de performance.

04

Génération et analyse des sorties expérimentales

Campagnes rejouables, CSV horodatés et pipeline de figures pour produire des artefacts traçables.

05

Synthèse et démonstration

Conclusion sur la robustesse du système, reproductibilité inter-plateforme et exécution finale.

Vue d'ensemble du système expérimental

Le système est conçu comme un moteur expérimental dédié à la comparaison cryptographique dans un cadre contrôlé.

Il automatise les campagnes de mesure, applique un protocole identique sur x86 et Raspberry Pi, puis produit des résultats fiables et traçables.

L'objectif est d'obtenir une analyse technique défendable de la performance et de la robustesse, plutôt qu'une application orientée utilisateur.

1

Nature et objectifs du système

01a

Comparer les algorithmes symétriques dans un cadre contrôlé, avec des critères mesurables et reproductibles.

2

Architecture logicielle en couches

01b

Séparation claire des responsabilités entre orchestration, moteur de chiffrement, modes opératoires et primitives cryptographiques.

3

Périmètre expérimental mesuré

01c

Comparaison menée sur x86 et Raspberry Pi, selon des métriques communes: débit, avalanche et stabilité (IC95).

4

Hypothèses et limites

01d

Analyse centrée sur la validité cryptographique et la performance mesurée; hors périmètre: UX, sécurité réseau et déploiement production.

Objectifs du projet

Comparer

5 algorithmes

Mesurer

2 plateformes

Valider

KAT + IC95

Performance comparée

Mesurer le débit de chiffrement et de déchiffrement sur deux architectures réelles x86 sous Windows et ARM sur Raspberry Pi. Conserver les mêmes jeux d'essai et les mêmes paramètres pour garantir une comparaison stable lisible et équitable.

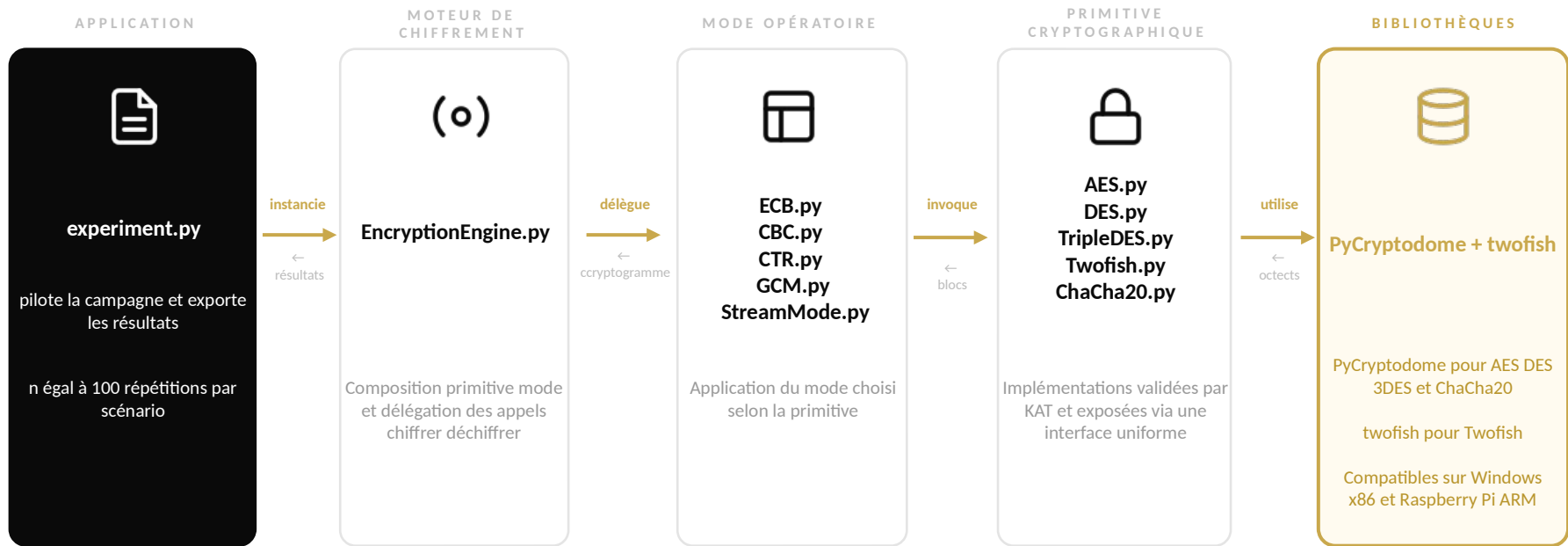
Robustesse cryptographique

Évaluer la diffusion cryptographique avec l'effet avalanche en observant le basculement d'un bit dans le message et dans la clé. Vérifier la stabilité statistique avec un intervalle de confiance à 95 pour soutenir des conclusions solides.

Reproductibilité expérimentale

Rejouer une chaîne complète et traçable qui couvre la campagne de mesures la production des résultats CSV la validation KAT et IC95 puis la génération des graphiques de synthèse. Obtenir des artefacts vérifiables qui facilitent la revue la comparaison entre plateformes et la défense des résultats.

Architecture logicielle en couches



Un protocole identique sur les deux plateformes permet d'attribuer les écarts observés au matériel avec une méthode de mesure constante et reproductible

Périmètre expérimental **mesuré**

	ALGORITHMES	AES • DES • 3DES • Twofish • ChaCha20	5 implémentations symétriques comparées dans le même cadre expérimental.
	PLATEFORMES	x86 Windows • ARM Raspberry Pi	Même protocole rejoué sur deux environnements matériels réels.
	MÉTRIQUES	Débit • Avalanche • IC95	Performance, diffusion cryptographique et stabilité statistique.
	SORTIES	CSV de résultats • Graphiques d'analyse	Résultats exportés dans un format traçable et comparable.

Le périmètre retenu couvre les objets comparés, les plateformes testées, les métriques observées et les sorties nécessaires à une analyse rigoureuse.

Hypothèses et limites

HYPOTHÈSES



- **Protocole identique**
Même logique d'exécution sur x86 et Raspberry Pi.
- **Jeux de tests harmonisés**
Paramètres et scénarios alignés pour préserver l'équité de comparaison dans le périmètre disponible.
- **Mesures répétées**
Campagnes répétées avec n égal à 100 puis export des résultats de façon traçable.
- **Validation de conformité**
Implémentations vérifiées par vecteurs de référence dans le protocole de validation.

LIMITES



- **Cadre expérimental ciblé**
Résultats valides dans le périmètre défini, sans généralisation absolue à tous les usages.
- **Dépendance au matériel**
Les performances observées dépendent des plateformes retenues et de leur contexte d'exécution.
- **Périmètre hors application**
L'étude n'évalue ni l'expérience utilisateur, ni le déploiement, ni la sécurité réseau.
- **Comparaison non universelle**
Tous les algorithmes ne partagent pas exactement les mêmes modes ou accélérations matérielles.

Les résultats doivent être lus comme une comparaison expérimentale contrôlée, valide dans son périmètre, et non comme une mesure universelle de tous les contextes d'usage.

Deux plateformes, un protocole rigoureux.

Le même protocole expérimental est appliqué sur x86 et Raspberry Pi afin d'établir une base de comparaison commune.

Matrice, paramètres statistiques et format de sortie sont harmonisés pour garantir la comparabilité des mesures.

Les écarts observés sont ensuite interprétés à protocole constant, en tenant compte du contexte matériel et système propre à chaque plateforme.

1

Deux plateformes, un protocole commun

02a

Protocole et base logicielle alignés entre x86 Windows et ARM Raspberry Pi 4; les écarts sont interprétés à protocole constant.

2

Matrice expérimentale structurée

02b

31 combinaisons algorithme-mode-clé, 5 tailles de message et 100 répétitions par configuration pour une base statistique homogène.

3

Environnements documentés et paramètres contrôlés

02c

Versions logicielles, paramètres d'exécution et conditions de mesure explicitement documentés sur les deux plateformes.

4

Conditions de reproductibilité

02d

Même matrice expérimentale, mêmes répétitions, même format de sortie et base logicielle alignée pour garantir des comparaisons x86/ARM cohérentes et interprétables.

5

Contrôle des biais et limites du protocole

02e

Biais réduits par répétitions et agrégation statistique (IC95); limites explicitées pour une interprétation rigoureuse des résultats.

x86 vs ARM — protocole commun, contexte matériel distinct

LAPTOP · WINDOWS 11

Intel Core i5-10300H

- OS : Windows 11
- RAM : 32 Go
- Architecture : x86-64



VS



RASPBERRY PI 4 · LINUX

ARM Cortex-A72

- OS : Linux (Raspberry Pi OS)
- RAM : 4 Go
- Architecture : ARM64

✓ **AES-NI disponible (Intel)**

Accélération matérielle du chiffrement AES

✗ **AES-NI absent (Cortex-A72)**

Performance AES dépendante des extensions disponibles et des backends logiciels

Même code, même protocole, matériel distinct: la performance brute doit être interprétée dans son contexte d'architecture.

Matrice expérimentale — structurée et comparable

100

répétitions
par configuration

31

combinaisons
algorithme • mode • clé

155

configurations mesurées
par plateforme

TAILLES DE MESSAGE

64 o

256 o

1 024 o

4 096 o

16 384 o

CALCUL

31 combinaisons × **5** tailles = **155** cas / plateforme

155 cas × 100 répétitions assurent une base statistique stable et comparable entre plateformes.

REPRODUCTIBILITÉ

- Protocole identique sur x86 et ARM
- Export CSV horodaté
- Validation KAT réalisée avant interprétation des mesures

À protocole constant, les écarts observés sont interprétés en fonction de la plateforme plutôt que de la méthode.

Matrice **algorithme × mode × clé**

	ECB	CBC	CTR	GCM	Stream	Tailles de Clé
AES Standard de référence	✓	✓	✓	✓	N/A	128/192/256
DES Algorithme historique	✓	✓	✓	N/A	N/A	64
3DES Usage retiré des recommandations récentes	✓	✓	✓	N/A	N/A	128/192
Twofish Finaliste AES	✓	✓	✓	N/A	N/A	128/192/256
ChaCha20 Chiffrement de flux	N/A	N/A	N/A	N/A	✓	256

✓ Testé — chiffrement par blocs standard (ECB, CBC, CTR)

□ GCM / Stream — paradigme distinct : authentification intégrée (GCM) ou chiffrement de flux (Stream)

□ N/A — mode structurellement incompatible avec cet algorithme

Conditions de reproductibilité

Ces quatre conditions rendent les comparaisons x86/ARM méthodologiquement cohérentes et interprétables.

Même matrice expérimentale

C1

La même EXPERIMENT_MATRIX est exécutée sur les deux plateformes: mêmes algorithmes, mêmes modes, mêmes tailles de clé et de message.

Même nombre de répétitions

C2

100 répétitions par configuration réduisent la variance et produisent un intervalle de confiance à 95% statistiquement stable sur les deux plateformes.

Même format de sortie CSV

C3

Les résultats sont exportés avec la même structure (colonnes, types, convention de nommage). Les fichiers produits restent directement comparables entre x86 et ARM.

Même base logicielle

C4

Base logicielle identique sur x86 et ARM, avec dépendances alignées. L'encodage des données (UTF-8) et le format CSV sont explicitement contrôlés pour garantir la cohérence entre Windows et Raspberry Pi OS.

Contrôle des biais et **limites du protocole**

BIAIS CONTRÔLÉS



- 100 répétitions par configuration
- Moyennes et IC95
- Protocole identique sur les deux plateformes
- Validation par vecteurs de référence préalable

LIMITES ASSUMÉES



- Dépendance au matériel
- Modes non parfaitement symétriques entre algorithmes
- Validité limitée au cadre testé
- Hors périmètre: UX, réseau, production

La validité des constats repose sur un protocole contrôlé et un périmètre explicitement limité.

Validation cryptographique, avant toute mesure de performance.

Cette section établit la conformité cryptographique des implémentations avant toute analyse de performance.

Elle présente le principe des KAT, l'exemple concret de vérification, la couverture retenue par suite, le pipeline d'exécution centralisé, puis les résultats consolidés du profil principal sur les deux plateformes.

1

Principe KAT et références normatives

02a

Définit le KAT et ancre la validation dans des références normatives reconnues.

2

Vecteur de test et principe de vérification

02b

Montre concrètement comment un cas KAT est exécuté et validé par comparaison bit à bit.

3

Couverture KAT retenue par suite

02c

Précise, pour chaque suite, ce qui est testé, pourquoi ce choix est retenu et ce qu'il permet de valider.

4

Pipeline de validation KAT

02d

Présente l'exécution centralisée de la campagne KAT, de la préparation des suites jusqu'au rapport final.

5

Résultats KAT — profil principal

02e

Résume les résultats de conformité du profil principal sur l'ensemble des suites et des plateformes.

Principe **KAT** et références normatives

Qu'est-ce qu'un Known Answer Test (KAT)

- Un KAT applique un vecteur de test prédéfini à l'algorithme.
- L'exécution se fait avec des paramètres fixes (clé, mode, IV/nonce selon le cas).
- La sortie obtenue est comparée bit à bit à la sortie de référence attendue.

Pourquoi c'est important ?

- Le KAT valide la conformité fonctionnelle avant toute mesure de performance, avec contrôles stricts des vecteurs Twofish.
- Il évite d'interpréter des résultats de vitesse sur une implémentation incorrecte.
- Il fournit une preuve traçable, alignée sur les références normatives (NIST, RFC, corpus Twofish) et vérifiable par hachage des artefacts.

Le KAT est un prérequis méthodologique avant toute mesure de performance.

SOURCES DE REFERENCE

● NIST FIPS 197

AES

● NIST SP 800-17

DES

● NIST SP 800-67 Rev.2

3DES

● NIST SP 800-38A/D

NIST SP 800-38A (ECB, CBC, CTR) / NIST SP 800-38D (GCM)

● RFC 8439

ChaCha20

● Schneier et al. (1998), Counterpane Systems

Twofish (ECB_VK, ECB_VT, ECB_TBL)

Couverture KAT retenue par famille

	Couverture par suite	Raison méthodologique	Garantie obtenue
AES	AES-128, AES-192 et AES-256	Couvrir les trois tailles de clé sans multiplier les cas redondants	Conformité sur les variantes principales de la suite AES
DES	Couples clair/chiffré représentatifs	Vérifier le comportement sur des cas normatifs sans reprendre tout le corpus	Conformité fonctionnelle de la primitive DES
3DES	Cas à 2 clés et à 3 clés (EDE)	Couvrir les deux configurations opérationnelles principales	Validité des variantes TDEA les plus représentatives
Modes	Un cas pour ECB, CBC et CTR	Isoler la logique de chaque mode sans surpondérer la famille	Conformité de l'enchaînement, IV et compteur selon le mode
GCM	Chiffrement, tag et vérification d'intégrité	Tester les fonctions essentielles AEAD, pas seulement le chiffrement	Conformité du traitement authentifié et de la vérification
ChaCha20	Vecteur RFC 8439 et validation via le wrapper	Couvrir la conformité normative et l'intégration dans le projet	Conformité du vecteur normatif et de son intégration dans l'implémentation
Twofish	Deux cas par famille VK, VT et TBL en mode ECB	Couvrir VK, VT et TBL avec un échantillonnage représentatif sans exécuter le corpus complet	Conformité vérifiée sur les familles VK, VT et TBL dans la campagne KAT actuelle

Cette slide décrit le périmètre retenu, pas une exhaustivité totale de tous les vecteurs.

Vecteur de test et principe de vérification

UN VECTEUR DE TEST ASSOCIE

- **Entrée connue** — texte clair, données brutes
- **Paramètres fixés** — clé, IV, nonce, mode, taille
- **Sortie attendue** — valeur de référence normative

LE PRINCIPE DE VÉRIFICATION

- 1 L'implémentation chiffre ou déchiffre l'entrée.
- 2 Le résultat obtenu est comparé bit à bit à la sortie attendue.
- 3 Une égalité exacte valide la conformité au vecteur de référence.

EXEMPLE CONCRET — AES-128 ECB

Entrée 00000000...00 (16 octets)

Clé 00000000...00 (16 octets)

Mode ECB

Attendu 66e94bd4ef8a2c3b884cfa59ca342b2e

Obtenu 66e94bd4ef8a2c3b884cfa59ca342b2e

Résultat ✓ PASS — conformité validée

Un vecteur ne mesure pas la vitesse — il vérifie que l'algorithme produit le bon résultat pour un cas de référence donné.

Pipeline de validation KAT



Orchestrateur principal: run_kat.py

PRÉPARE



Rôle: sélectionner le profil, les suites, et charger les vecteurs.

Mécanisme: lecture de la configuration KAT et préparation des jeux de test.

Fichiers sources:

- kat_aes.py
- kat_des.py
- kat_3des.py
- kat_modes.py
- kat_gcm.py
- kat_chacha20.py
- kat_twofish.py



EXÉCUTE



Rôle: exécuter chaque vecteur KAT par suite.

Mécanisme: appel séquentiel des modules KAT et calcul des sorties.

Sortie: résultats bruts par vecteur.



VÉRIFIE



Rôle: comparer attendu vs obtenu, puis PASS/FAIL.

Mécanisme: comparaison bit à bit attendu vs obtenu + assertion.

Fichiers de vérification:

- kat_aes.py
- kat_modes.py
- kat_gcm.py
- kat_twofish.py

Sortie: statut par vecteur et détails d'échec.



RAPPORTE



Rôle: agréger et afficher le total par suite + statut global.

Mécanisme: agrégation par suite et par plateforme, avec profil affiché.

Exemple de sortie: kat_results.txt

Sortie: total global et rapport final de conformité (profil core/full affiché).

Résultats KAT — profil principal

SUITE	SOURCE	COUVERTURE	ASSERTIONS	RÉSULTAT x86 & ARM
AES	NIST FIPS 197	Vecteurs couvrant les clés 128, 192 et 256 bits	10	✓ PASS
DES	NIST SP 800-17	Couples clair/chiffré représentatifs issus du corpus KAT	18	✓ PASS
3DES	NIST SP 800-67	Cas représentatifs à 2 clés et à 3 clés (EDE)	4	✓ PASS
Modes	NIST SP 800-38A	Un cas représentatif pour chaque mode opératoire : ECB, CBC et CTR	6	✓ PASS
GCM	NIST SP 800-38D	Chiffrement authentifié, tag et vérification d'intégrité	10	✓ PASS
ChaCha20	RFC 8439	Vecteur normatif RFC 8439 et validation via le wrapper	6	✓ PASS
Twofish	Schneier et al.	Un vecteur représentatif par famille : VK, VT et TBL	6	✓ PASS
TOTAL			60	100% PASS

Génération et analyse des sorties expérimentales.

Cette section montre comment les sorties sont produites et validées de bout en bout: génération des données, pipeline de graphiques, contrôle statistique IC95 (théorie puis application), puis validation cryptographique par l'effet d'avalanche.

1

Pipeline de génération des données expérimentales

04a

Présente la chaîne de production des mesures et l'export CSV structuré.

2

Pipeline de génération des graphiques

04b

Montre comment les CSV sont transformés en figures via préparation, construction et style commun.

3

IC95 relatif : théorie et lecture pratique

04c

Explique la formule, les paramètres statistiques et les repères d'interprétation.

4

IC95 relatif : application des gates

04d

Applique G1-G2-G3 sur les résultats réels et produit un verdict CI publier ou bloquer.

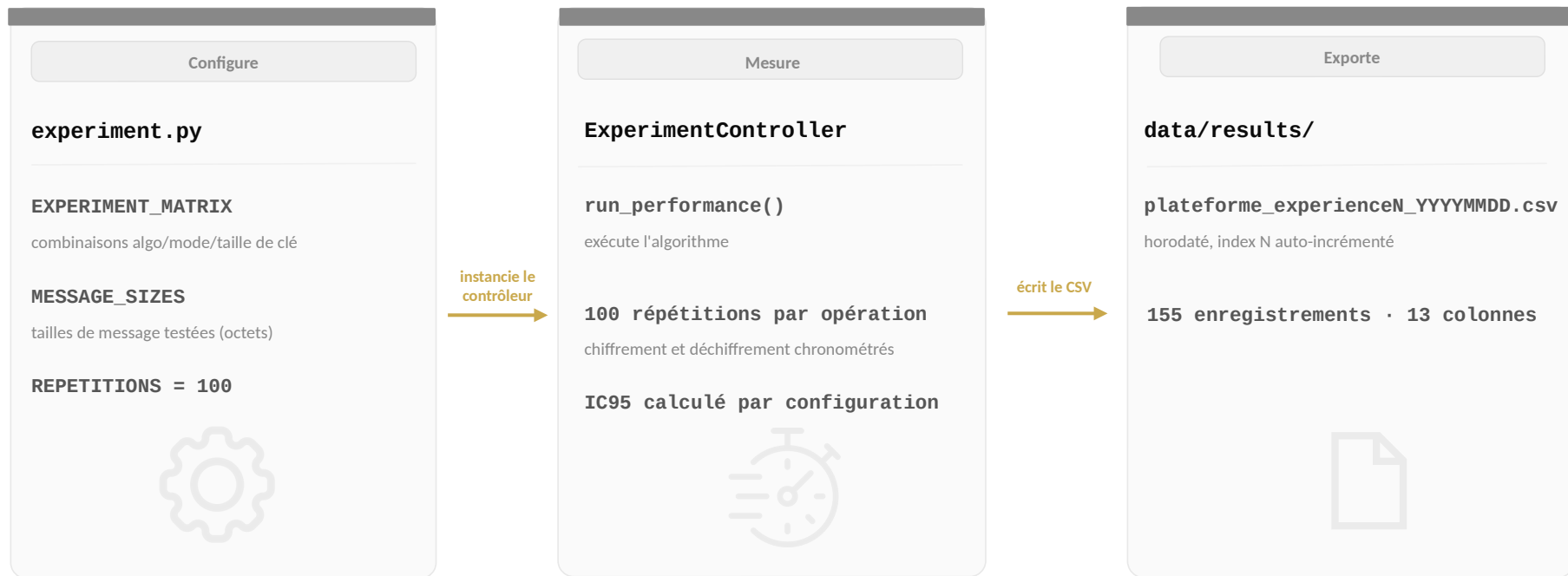
5

Validation des résultats : effet d'avalanche

04e

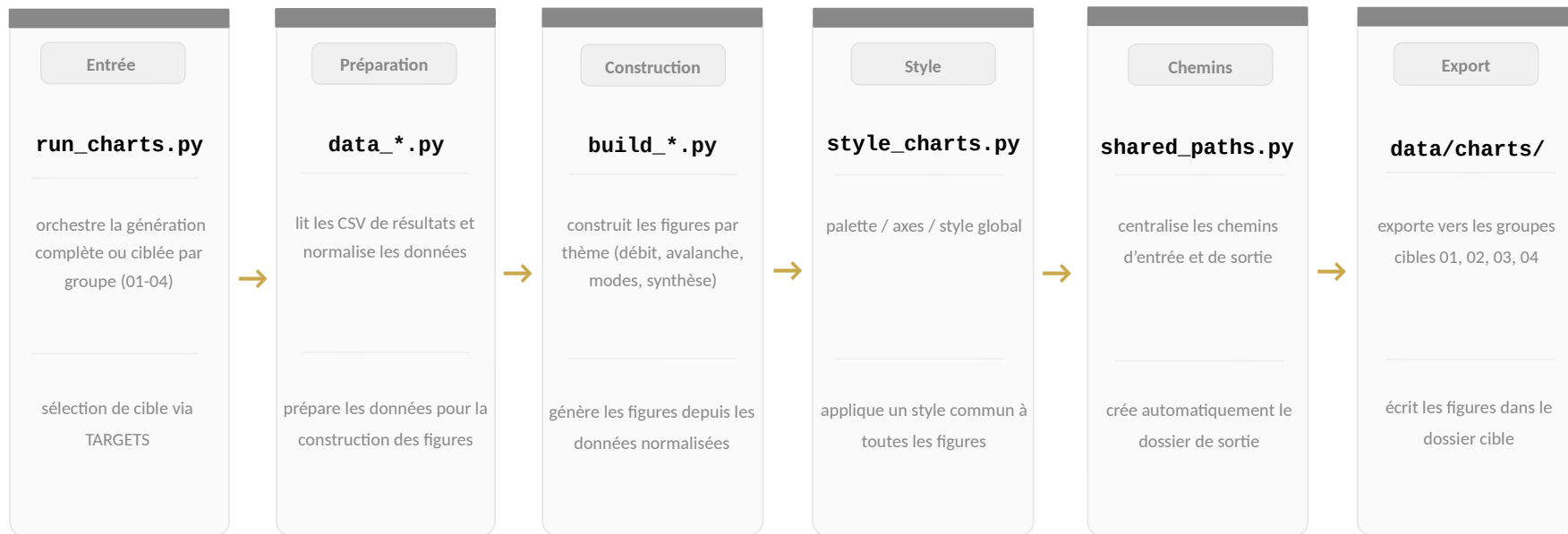
Vérifie la cohérence des résultats avec la littérature et la stabilité entre plateformes.

Pipeline de génération des données expérimentales



Un protocole identique sur les deux plateformes garantit la comparabilité des mesures exportées.

Pipeline de génération des sorties graphiques



commande unique, pipeline complet et export structuré

IC95 relatif : théorie et lecture pratique

FORMULES

$$IC95 (n \geq 30) \approx 1,96 \times \sigma \sqrt{n}$$

$$IC95 \text{ relatif } (\%) = \frac{ci95_encrypt_mbps}{throughput_encrypt_mbps} \times 100$$

Paramètres utilisés : n = 100 · confiance = 95 % · z = 1,96

INTERPRÉTATION

**≤ 5 %**

Variabilité faible — résultat fiable

Très bon**5-10 %**

Variabilité modérée — interprétation prudente

Acceptable**> 10 %**

Variabilité élevée — investigation requise

Bruit élevé

** Seuils 5/10 % = repères de lecture; la décision CI vient des gates G1-G3.*

**100**

répétitions / configuration

**95 %**

niveau de confiance

**1,96**

Facteur z (approx. normale)

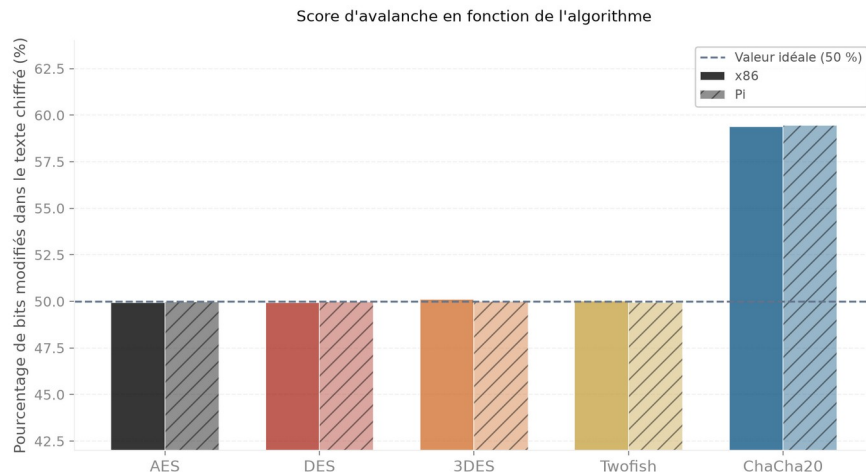
IC95 relatif : application des gates



✓ **PASS** 3/3 gates validés · publication CI autorisée

Validation des résultats : effet d'avalanche

Ce graphique vérifie la cohérence expérimentale avec la littérature: les block ciphers doivent converger vers ~50% d'effet d'avalanche, et les résultats doivent rester stables entre x86 et ARM. Nos mesures respectent ces deux critères.



Référence théorique (~50%)

AES, DES, 3DES et Twofish convergent vers 50% (écart moyen < 0,1 point), conforme aux critères de diffusion classiques.

Robustesse inter-plateformes

Sur les chiffrements par blocs, l'écart x86/ARM est négligeable, ce qui confirme une stabilité inter-plateformes élevée.

Profil ChaCha20 (ARX)

ChaCha20 est mesuré autour de 59,4-59,5% sur les deux plateformes. Ce profil plus élevé est cohérent avec une diffusion ARX et reste stable entre x86 et ARM.

Interprétation: cohérence théorique + stabilité inter-plateformes = résultats crédibles.

Références: Paar & Pelzl, Understanding Cryptography (2010), section 3.3; RFC 8439 (ChaCha20)

Synthèse et démonstration

Cette section présente la démonstration finale de fiabilité du pipeline expérimental.

Validation logicielle complète, chaîne d'exécution, preuve de reproductibilité et conclusion. Les résultats sont techniquement validés, traçables et directement exploitables pour l'analyse finale (TN4).

1

Validation complète du pipeline expérimental

05a

Contrôles logiciels, cryptographiques et statistiques — le pipeline respecte l'ensemble des critères de validation.

2

Scripts d'exécution

05b

Chaîne d'exécution cohérente, de la validation jusqu'à la génération des artefacts.

3

Preuve et reproductibilité des résultats

05c

Protocole formalisé et dossier de preuves liant les résultats au commit soumis — traçabilité vérifiée.

4

Conclusion générale

05d

L'ensemble des validations confirme que les résultats sont robustes et prêts pour l'analyse TN4.

Validation complète du pipeline expérimental

120 tests exécutés

100% couverture lignes + branches

Seuil appliqué automatiquement par pytest, avec échec CI si la couverture lignes ou branches descend sous 100%

PÉRIMÈTRE COUVERT



- domain
- application
- validation
- scripts/audit
- tests différentiels contre des implémentations de référence



CE QUE ÇA GARANTIT

- Cœur logiciel validé par tests automatisés
- Conformité crypto validée par KAT
- Contrôles statistiques IC95 validés avec gates qualité validées
- Régressions bloquées automatiquement en CI

Les critères de validation logicielle sont satisfaits et le pipeline est techniquement prêt pour l'analyse TN4.



Scripts d'exécution

À exécuter depuis `crypto-experiments` (environnement virtuel activé)

VALIDE

01

`python scripts/run_kat.py`

- Exécute les suites KAT
- Vérifie la conformité des implémentations
- Un PASS complet valide la base cryptographique

MESURE

02

`python scripts/experiment.py`

- Lance la matrice expérimentale
- Mesure toutes les configurations
- Exporte les CSV horodatés dans `data/results/`

GÉNÈRE

03

`python scripts/run_charts.py`

- Lit les CSV de `data/results/`
- Génère les figures via les modules `build_*`
- Exporte les graphiques dans `data/charts/`

Validation statistique complémentaire: IC95 et gates appliqués après le benchmark (CI)

Preuve et reproductibilité des résultats

01 Protocole de validation formalisé

`INF1430-TN4-validation-proof-protocol.md`

Une étape supplémentaire vers la préparation TN4 a été franchi, avec des résultats appuyés par un protocole reproductible et un dossier de preuves lié au commit soumis.

02 Générateur de bundle automatisé

`audit_bundle.py`

Le script de génération de preuves capture automatiquement les journaux d'exécution, les artefacts et le hachage du commit au moment de la génération.

03 Dossier de preuves généré et vérifié

Le dossier de preuves final a été généré avec succès. Le hachage du manifeste correspond exactement à celui du commit soumis, avec intégrité confirmée.

04 Validation finale réussie

Les trois contrôles exécutés dans le dossier de preuves sont conformes: pytest sans échec, KAT entièrement validé et audit IC95 validé.



Maturité du processus : Les fichiers de référence Twofish (ECB_VK/VT/TBL) sont protégés par des fichiers .sha256 et le CI vérifie l'intégrité des données de test avant chaque exécution KAT.

Les résultats sont maintenant traçables et reproductibles, avec une preuve liée au commit soumis

Conclusion **générale**

Le système est opérationnel sur x86 et ARM et validé par tests + KAT (AES, DES/3DES, ChaCha20, modes, Twofish).

Avec un protocole identique sur les deux plateformes, les mesures génèrent des artefacts comparables et reproductibles; les conclusions sont robustes dans le périmètre testé.

De la validation cryptographique à la mesure comparative, le protocole reste identique entre x86 et ARM, et les résultats sont reproductibles.

Validation démontrée

- ✓ Même socle de tests et KAT sur x86 et ARM; exécution validée.

Mesures reproductibles

- ✓ Même protocole, mêmes paramètres, mêmes scripts sur x86 et ARM.

Prêt pour l'analyse

- ✓ Les sorties alimentent directement le pipeline de génération des figures.